

Basis Data Lanjutan

Pertemuan Jobsheet 7



Disusun Oleh:

Nabila Nur 'Abidah Putri Valiandra

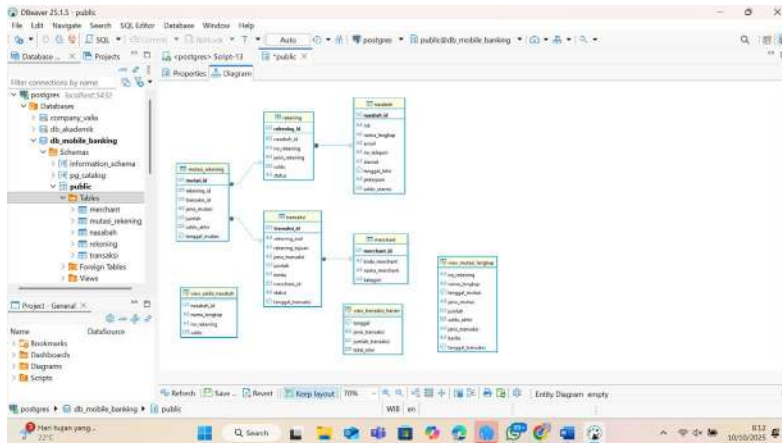
244107060086

D-IV Sistem Informasi Bisnis

2C

Pertanyaan 1

a. Screenshot hasil diagram yang sudah ada, dan berikan hasil analisis kalian mengenai desain basis data pada db_mobile_banking



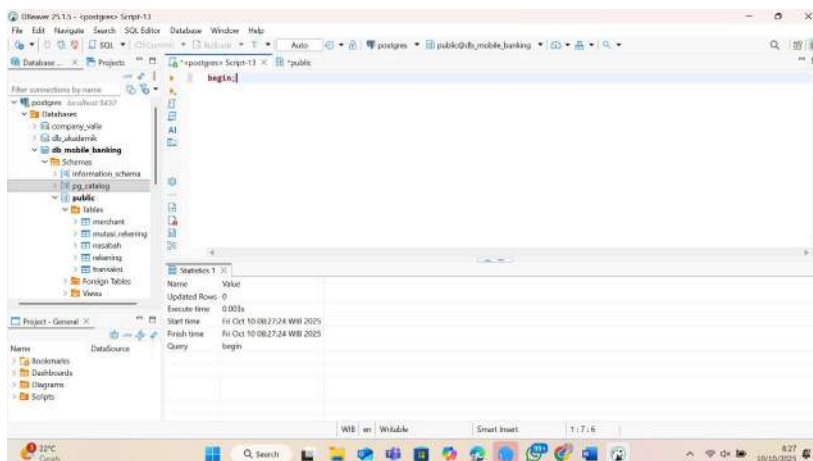
Analisis: Desain basis data db_mobile_banking menunjukkan sistem mobile banking yang mengelola data untuk nasabah, rekening, transaksi, dan mutase saldo. Lalu untuk tiap table memiliki fungsi yang jelas, misalnya pada table nasabah untuk data pengguna, rekening untuk informasi akun bank, transaksi untuk perubahn saldo. Selain itu ada beberapa view seperti view_salso_nasabah, view_transaksi_haran, dan view_mutasi_lengkap.

b. jelaskan pula relasi yang ada

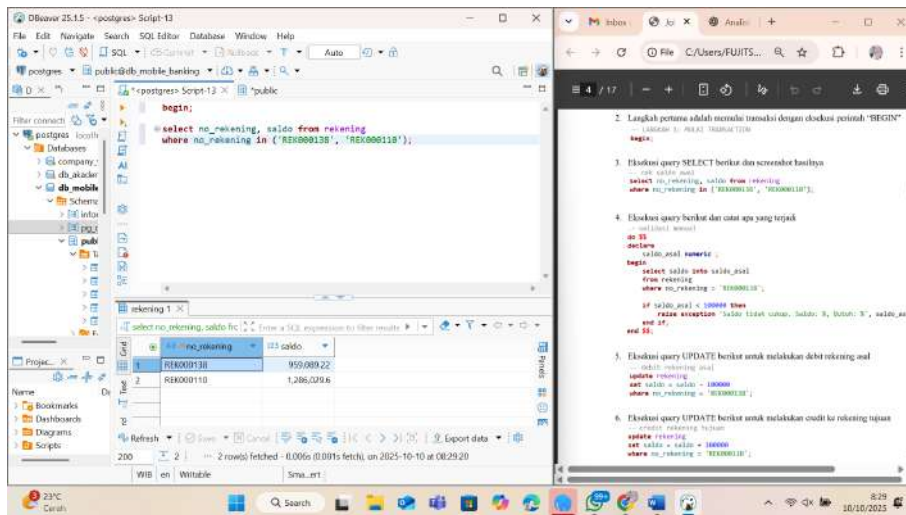
- nasabah ke rekening (satu nasabah bisa punya banyak rekening 1:N)
- rekening ke mutase_rekening (satu rekening bisa memiliki banyak mutase 1:N)
- transaksi ke mutase_rekening (setiap transaksi melibatkan rekening asal dan tujuan (N:1))
- transaksi ke merchant (transaksi bisa terhubung ke satu merchant N:1)

Praktikum 1

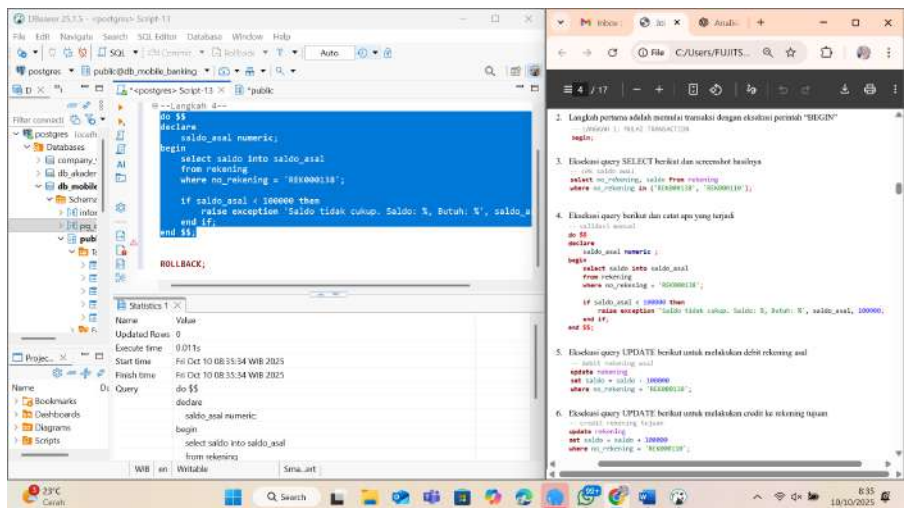
Langkah 2



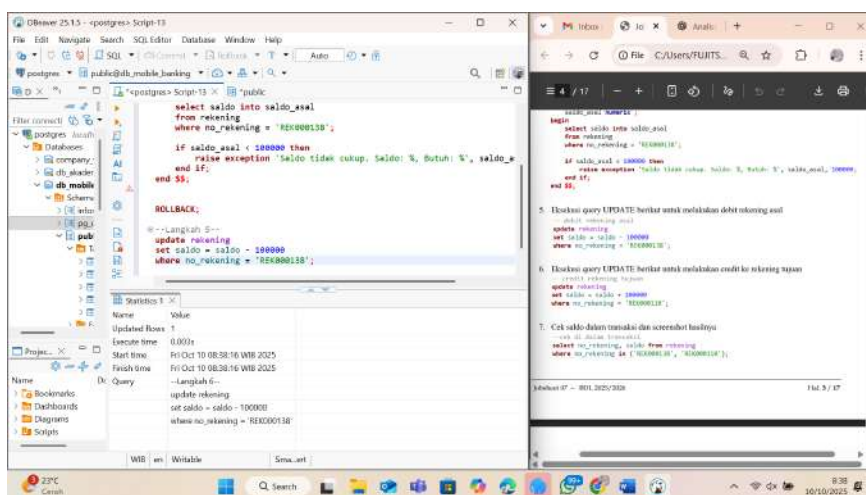
Langkah 3



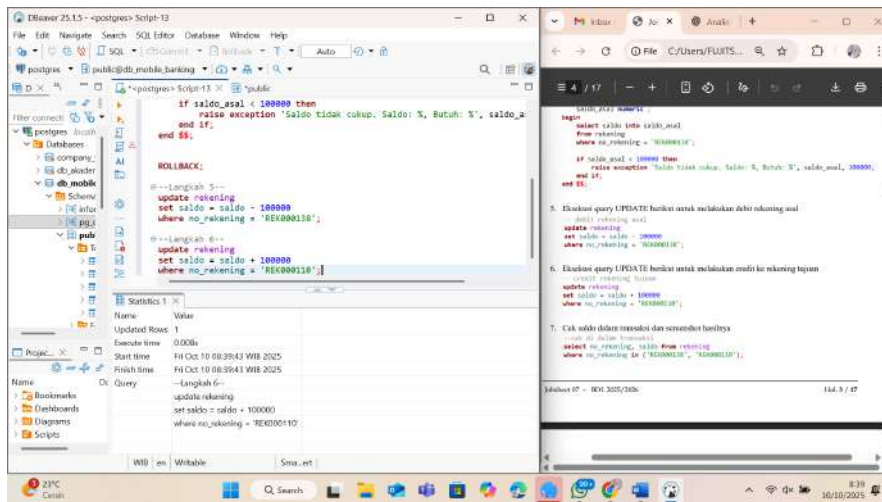
Langkah 4



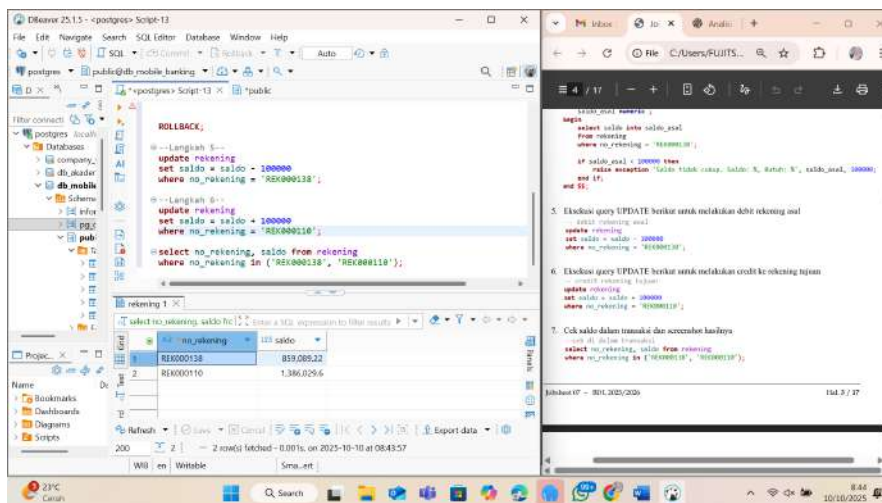
Langkah 5



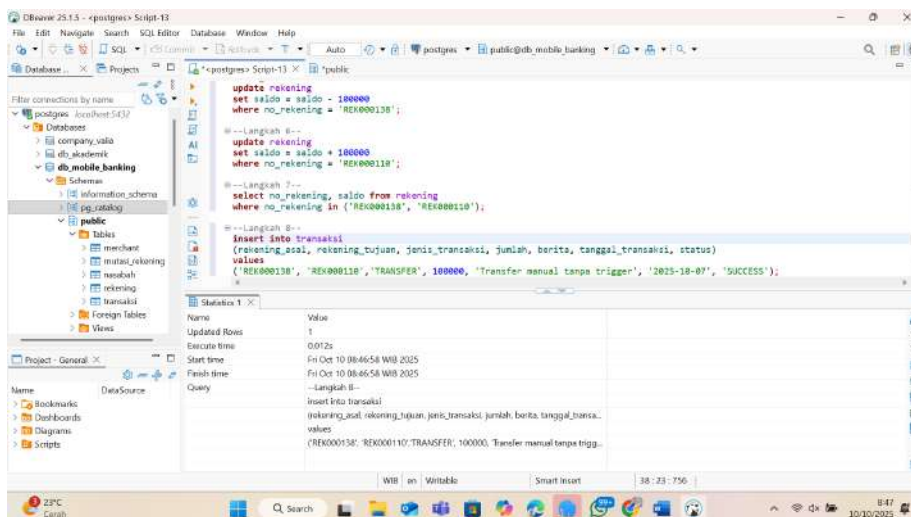
Langkah 6



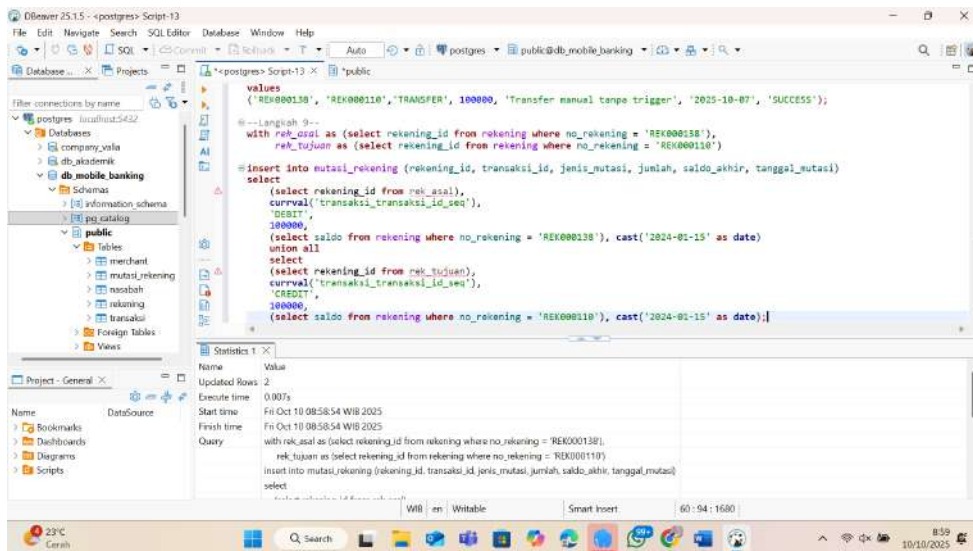
Langkah 7



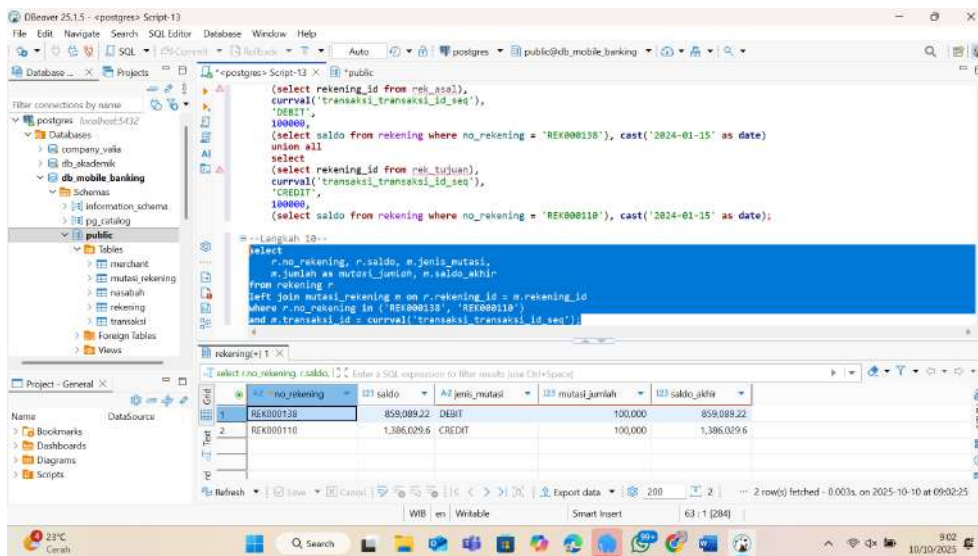
Langkah 8



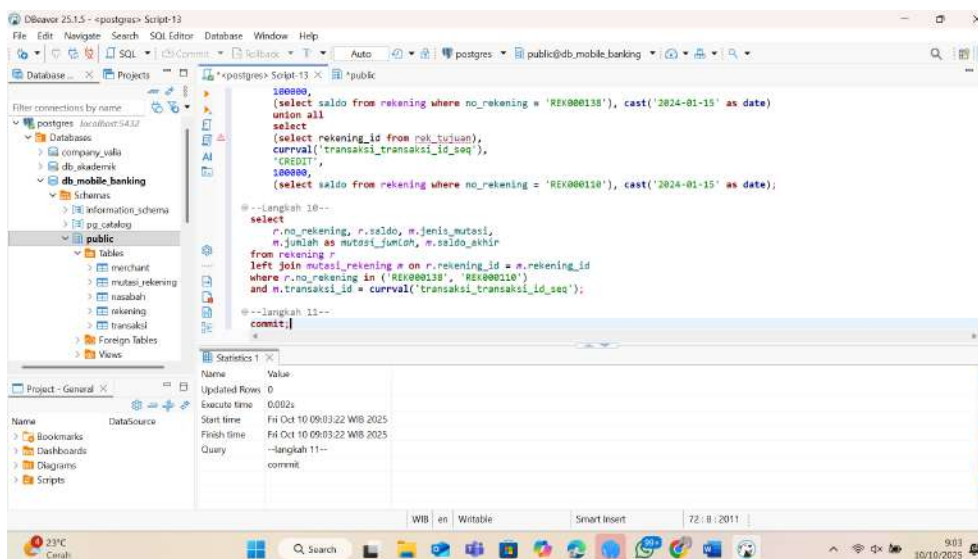
Langkah 9



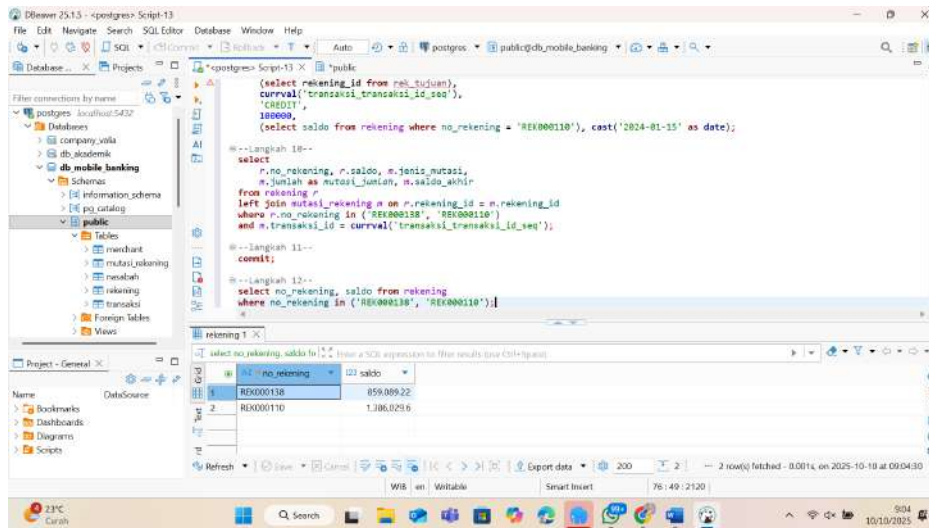
Langkah 10



Langkah 11



Langkah 12



Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
3	Cek saldo awal	REK000138: 959,089.22 REK000110: 1,286,029.6		Menampilkan saldo awal dari REK000138 dan REK000110
7	Cek dalam transaksi		REK000138: 859,089.22 REK000110: 1,386,029.6	Menampilkan saldo setelah dilakukan transaksi debit dan credit, saldo untuk rek000138 akan berkurang 10000 dan saldo di rek000110 akan bertambah 10000
10	Cek mutasi		Debit: 10000 Credit: 10000	Menampilkan debit dan credit yang dilakukan
12	Cek setelah commit		REK000138: 859,089.22 REK000110: 1,386,029.6	Setelah di commit saldo rekening akan sama seperti langkah 7

Pertanyaan Analisis

a. Apa yang terjadi jika berhenti di langkah 7 tanpa COMMIT?

Jawaban: maka data yang tersimpan akan menampilkan sama seperti langkah 3. Karena commit itu kayak menyimpan secara sementara, kalau tidak di commit maka outputnya tidak ada perubahan.

b. Bagaimana jika terjadi error di langkah 4 atau 5?

Jawaban: tidak terjadi error.

c. Sekarang eksekusi perintah ROLLBACK, Apa yang terjadi jika menjalankan ROLLBACK setelah COMMIT?

Jawaban: Tidak terjadi apa-apa dan akan tetap dalam kondisi terakhir setelah kita commit tadi.

Praktikum 02 – SAVEPOINT

Langkah 3

The screenshot shows the DBeaver SQL editor with a query executed. The query is:

```
--Langkah 3--
begin;
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000160', 'REK000095');
```

The results are displayed in a table:

no_rekening	saldo
REK000160	8242,20423
REK000098	4,048,211.55
REK000149	14,262,793.49
REK000095	2,817,820.19
REK000160	10,284,349.61

Langkah 4

Operation 1

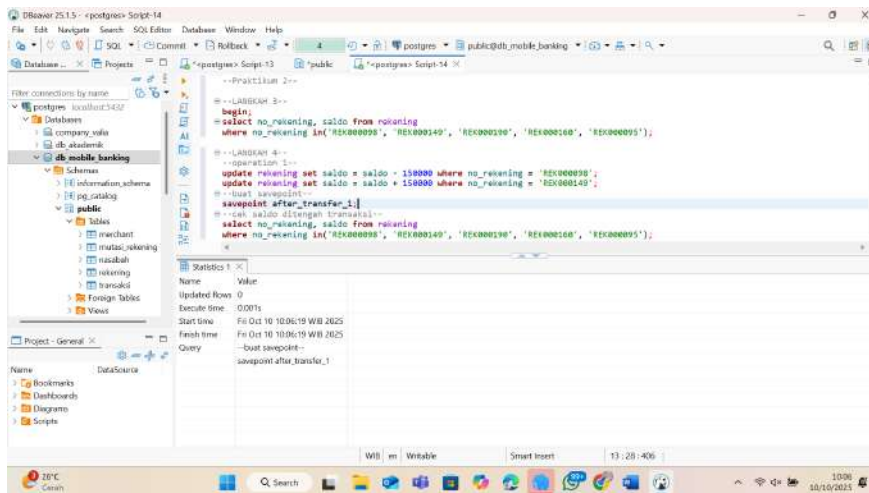
The screenshot shows the DBeaver SQL editor with a transaction containing several operations. The SQL script is:

```
--Praktikum 2--
--Langkah 3--
begin;
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000160', 'REK000095');
--Operation 4--
update rekening set saldo = saldo + 150000 where no_rekening = 'REK000098';
update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';
savepoint after_transfer_1;
--cek saldo ditengah transaksi--
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000160', 'REK000095');
```

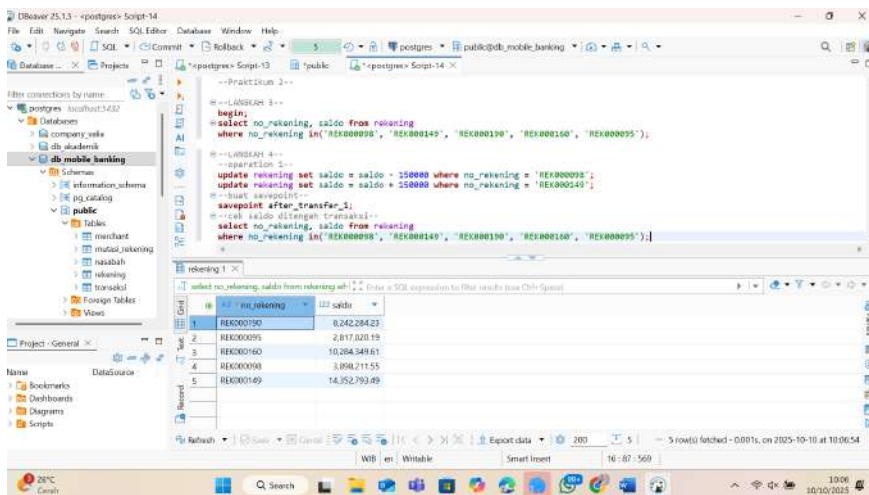
The Statistics window shows the execution details:

Name	Value
Updated Rows	2
Execute time	0.000s
Start time	Fri Oct 10 10:05:46 WIB 2025
Finish time	Fri Oct 10 10:05:46 WIB 2025
Query	update rekening set saldo = saldo + 150000 where no_rekening = 'REK000098'; update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';

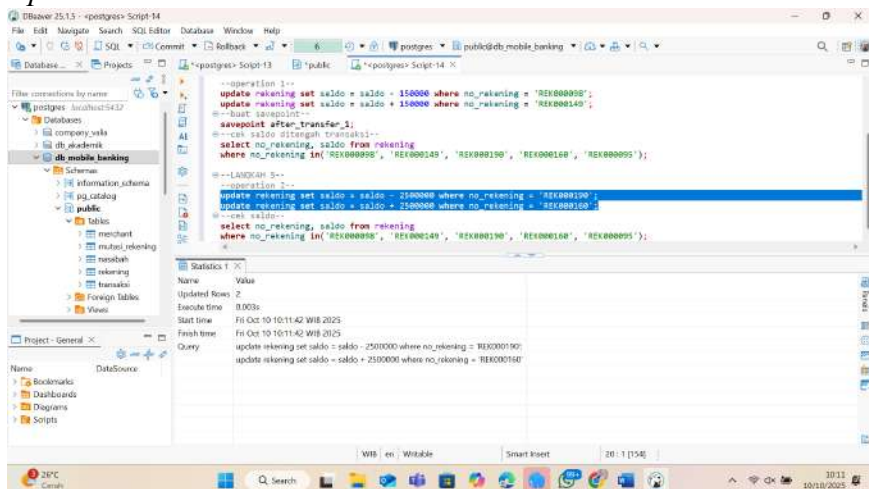
Savepoint



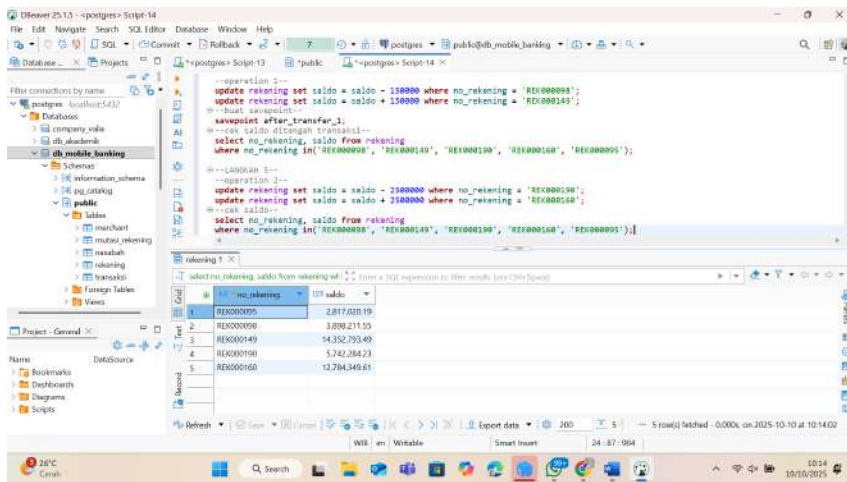
Cek saldo



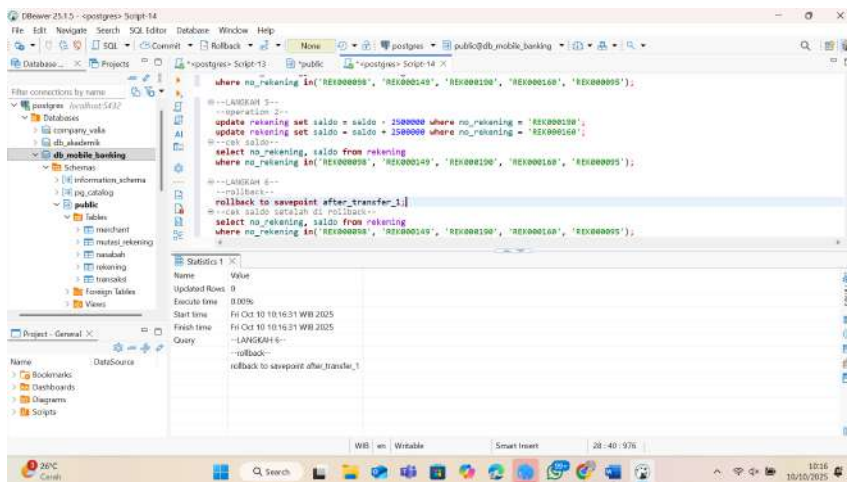
Langkah 5 Operation 2



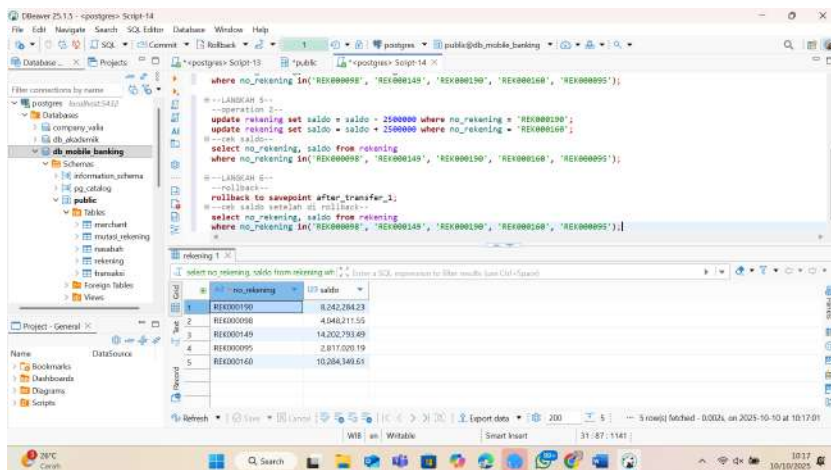
Cek Saldo



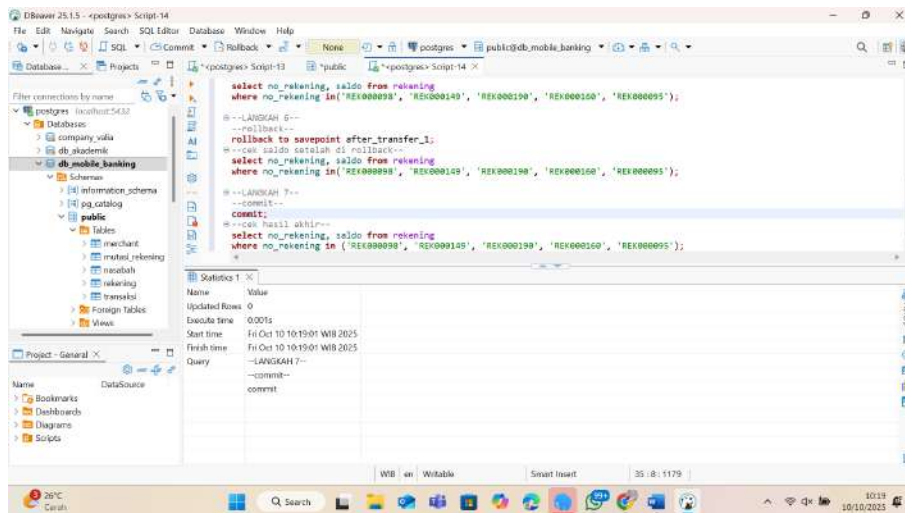
Langkah 6 Rollback



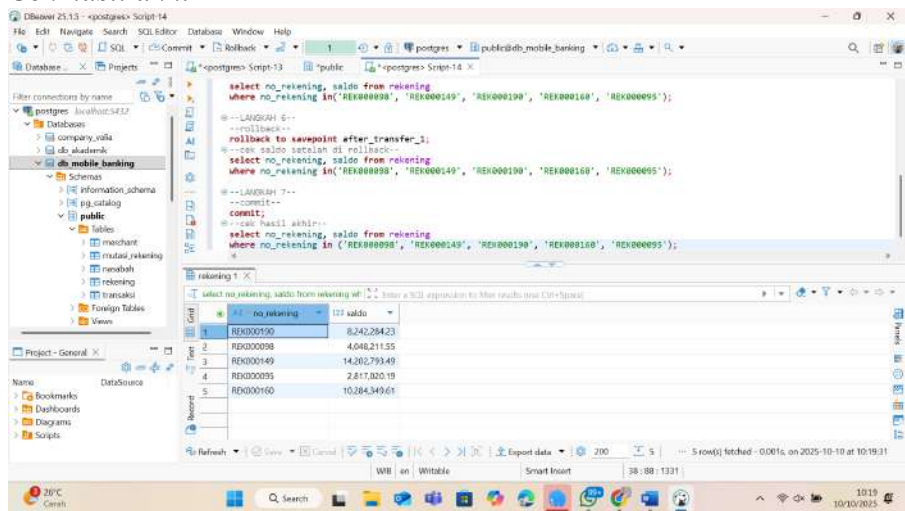
Cek saldo setelah di rollback



Langkah 7 Commit



Cek hasil akhir



Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
3	Cek saldo awal	REK000098: 4,048,211.55 REK000149: 14,202,793.49 REK000160: 10,284,349.61 REK000095: 2,817,020.19		Saldo awal dari tiap rekening dicatat sebelum dilakukannya transaksi
4	Cek setelah savepoint 1		REK000098: 3,898,211.55 REK000149: 14,352,793.49 REK000160: 10,284,349.61 REK000095: 2,817,020.23	Terjadi transaksi transfer yakni query update sebesar 150.000 dari REK000098 ke REK000149 (kredit). Saldo rekening asal berkurang 150.000

				dan saldo tujuan bertambah 150.000
5	Cek setelah operation 2		REK000098: 3,898,211.55 REK000149: 14,352,793.49 REK000160: 12,784,349.61 REK000095: 2,817,020.19	Terjadi transaksi transfer sebesar 2.500.000 dari REK000190(debit) ke REK000160(kredit). Saldo rekening asal berkurang 2.500.000 dan saldo rekening tujuan bertambah 2.500.000
6	Cek setelah rollback to savepoint 1		REK000098: 4,048,211.55 REK000149: 14,202,793.49 REK000160: 10,284,349.61 REK000095: 2,817,020.19	Perubahan dari langkah 5 dibatalkan karena rollback ke savepoint maka saldo kembali ke kondisi setelah savepoint 1
7	Cek setelah commit		REK000098: 4,048,211.55 REK000149: 14,202,793.49 REK000160: 10,284,349.61 REK000095: 2,817,020.19	Semua perubahan yang tersisa disimpan permanen setelah di commit

Pertanyaan Analisis

a. Bagaimana jika terjadi error di langkah 5?

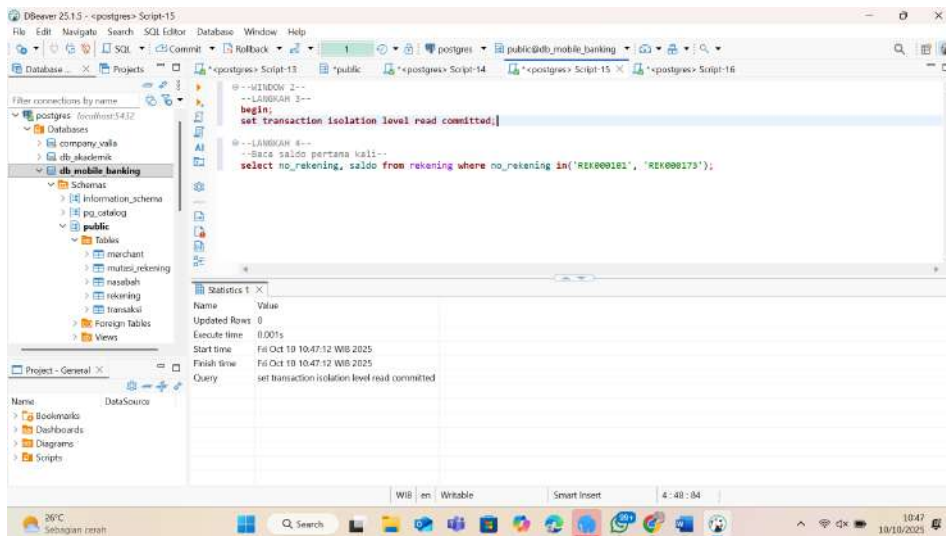
Jawaban: Tidak terjadi error.

b. Bagaimana jika kita menggunakan ROLLBACK tanpa menyebut savepoint?

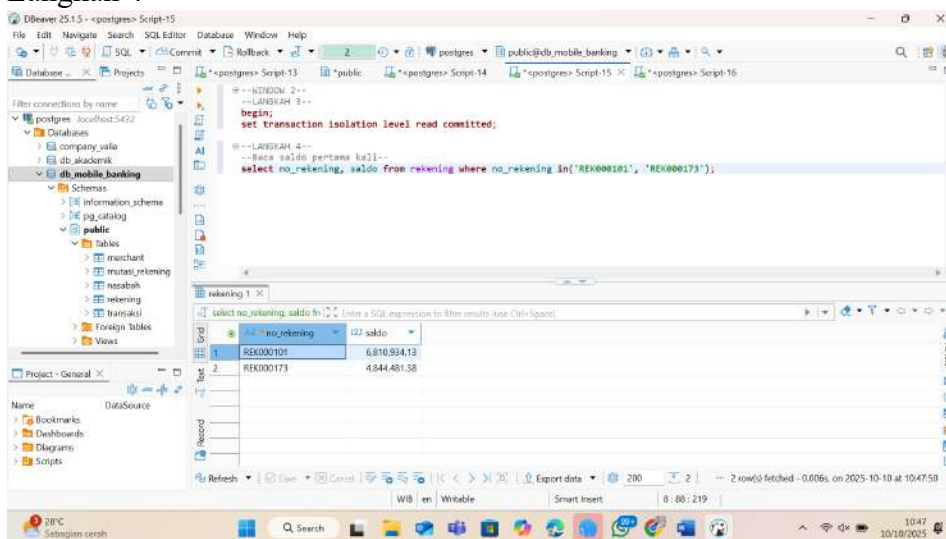
Jawaban: maka semua perubahan yang ada dalam transaksi akan dibatalkan total dan saldo rekening akan kembali ke kondisi sebelum transaksi dimulai yaitu sebelum BEGIN.

Praktikum 03 – Concurrent Transaction

Langkah 3

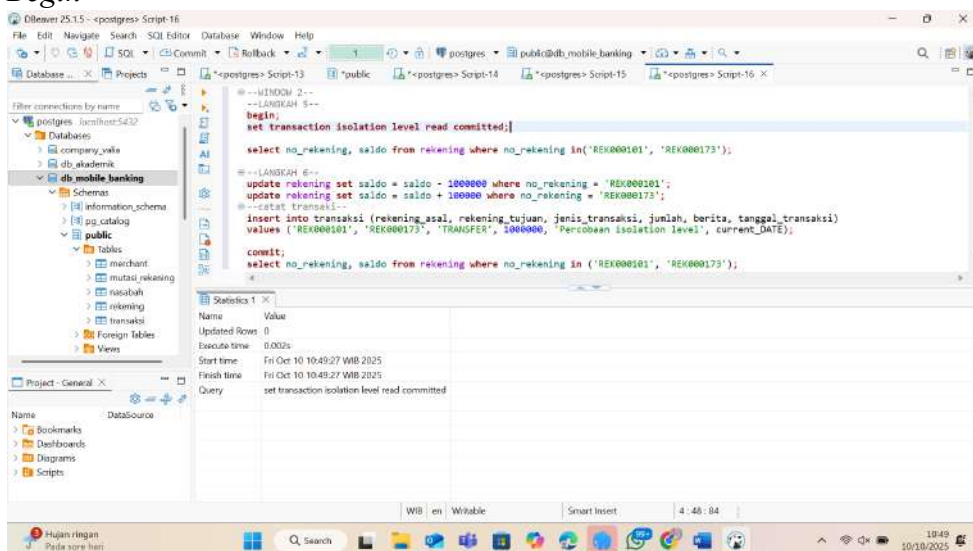


Langkah 4

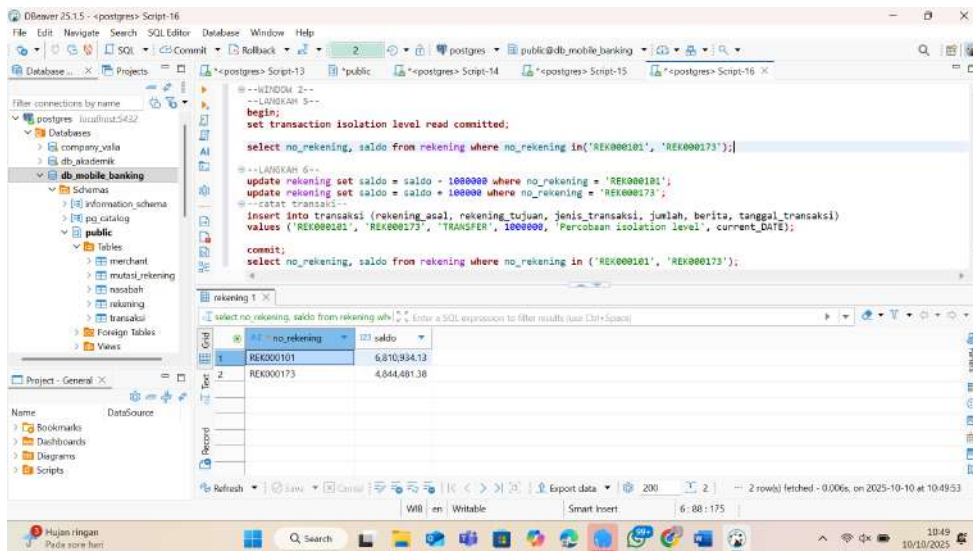


Langkah 5

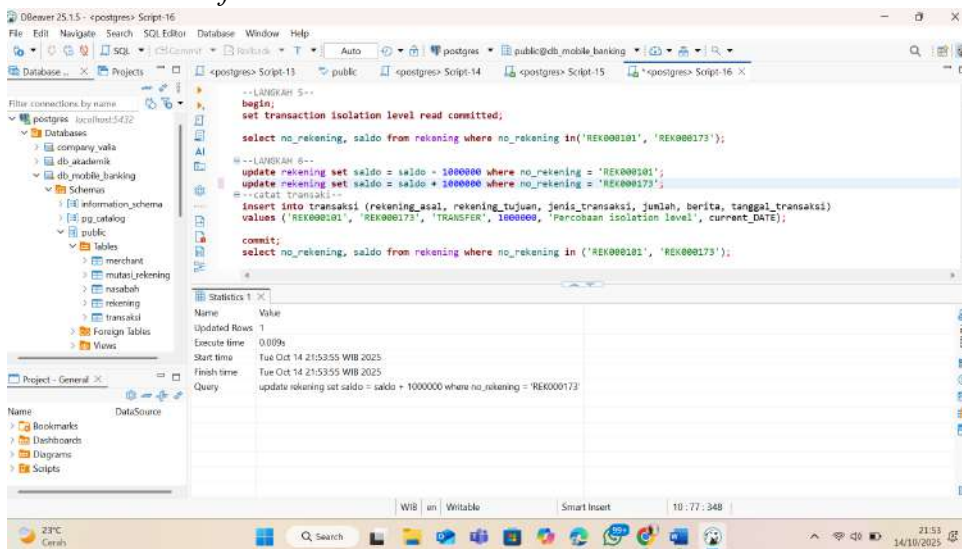
Begin



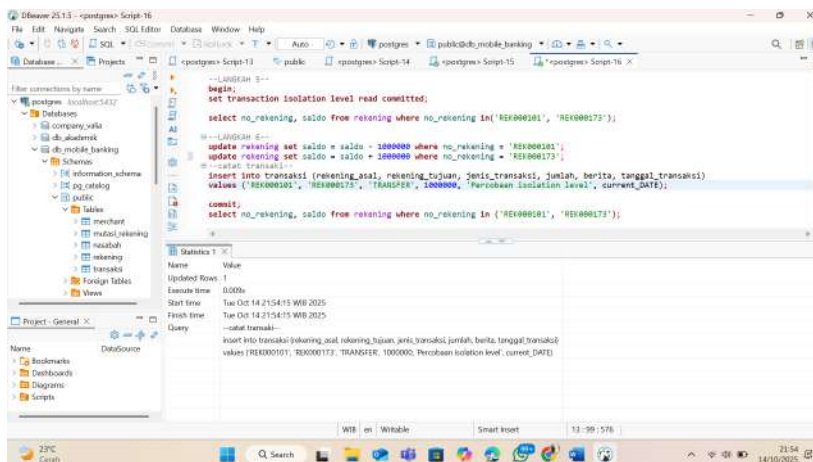
Select



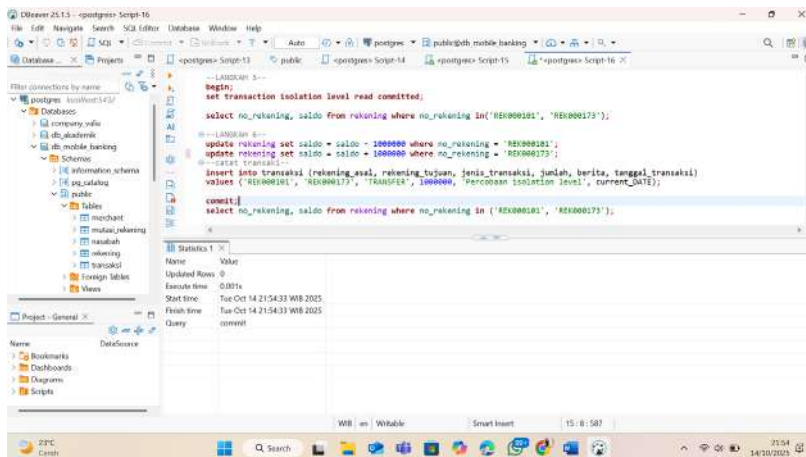
Langkah 6 Melakukan transfer



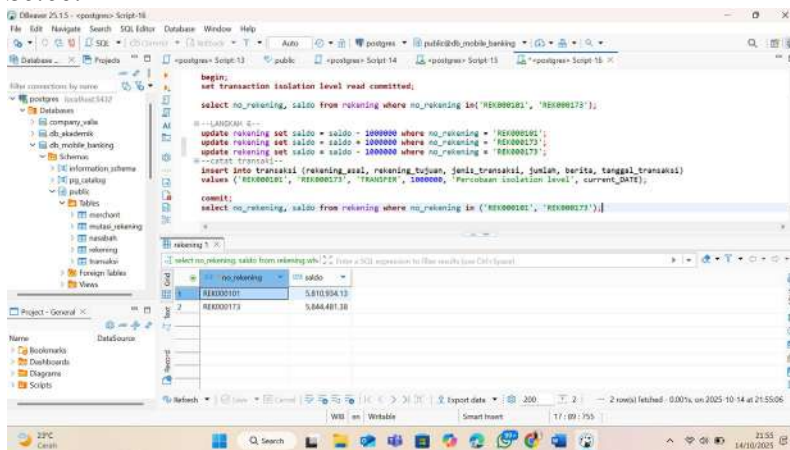
Mencatat transaksi



Commit

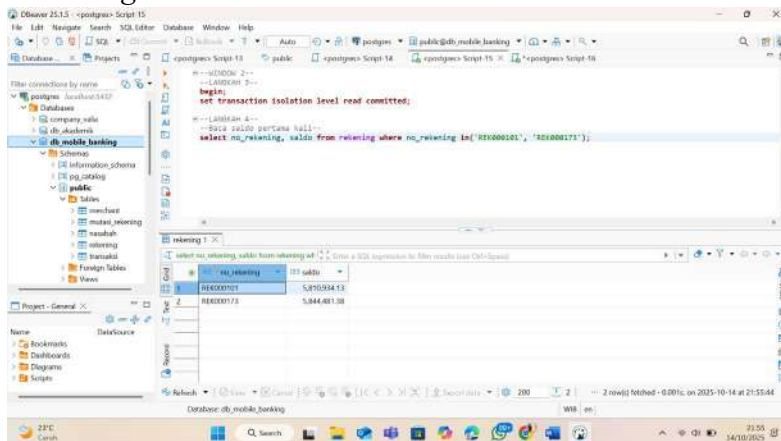


Select



Langkah 7

Cek ulang saldo di window 1



8. Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
4	Cek saldo awal window 1	REK000101: 6,810,934.13 REK000173: 4,844,481.38		Saldo awal tiap rekening dicatat sebelum dilakukan transaksi di window pertama.

5	Cek saldo awal window 2		REK000101: 6,810,934.13 REK000173: 4,844,481.38	Window kedua membaca saldo yang sama karena isolation level read committed. Tidak ada perubahan yang terlihat dari transaksi lain yang belum dicommit.
6	Cek saldo setelah commit window 2		REK000101: 5,819,934.13 REK000173: 5,844,481.38	Menampilkan debit dan credit sebesar 1000000 dari REK000101(debit) ke REK000173(kredit) lalu dicatat ke dalam table transaksi. Perubahan saldo menjadi permanen setelah kita melakukan commit.
7	Cek saldo akhir window 1		REK000101: 5,810,934.13 REK000173: 5,844,481.38	Setelah di commit saldo rekening akan sama seperti di semua window karena commit membuat perubahan secara permanen dan terlihat oleh transaksi yang berikutnya.

9. Pertanyaan Analisis

a. Bagaimana saldo awal dan akhir yang dilihat oleh transaksi pada window 1?

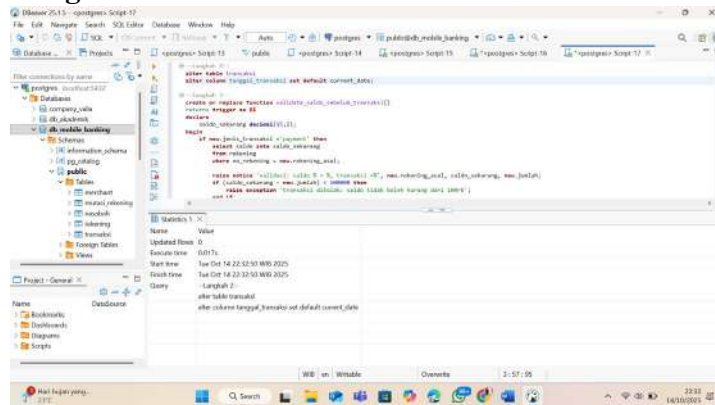
Jawaban: Saldo awal yang kita lihat di window 1 adalah pada REK000101: 6,810,934.13 dan pada REK000173: 4,844,481.38. Lalu kita melakukan transaksi transfer sejumlah 1000000 lalu kita commit, maka saldo akhir yang kita lihat di window 1 akan menjadi REK000101: 5,810,934.13 dan pada REK000173: 5,844,481.38. Sehingga pada window 1 kita dapat melihat perubahan yang dibuat setelah kita melakukan commit pada transaksi yang kita lakukan.

b. Apa yang dapat kamu simpulkan dari isolation level READ COMMITTED?

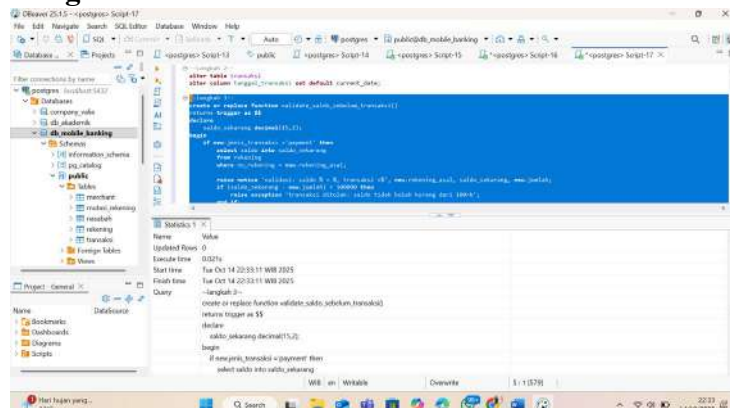
Jawaban: Isolation level read committed hanya akan membaca data yang sudah di commit, jadi pada transaksi yang ada di window 1 tidak akan melihat perubahan yang belum di commit dari transaksi lainnya, sehingga mencegah terjadinya dirty read .

Praktikum 04 – Trigger

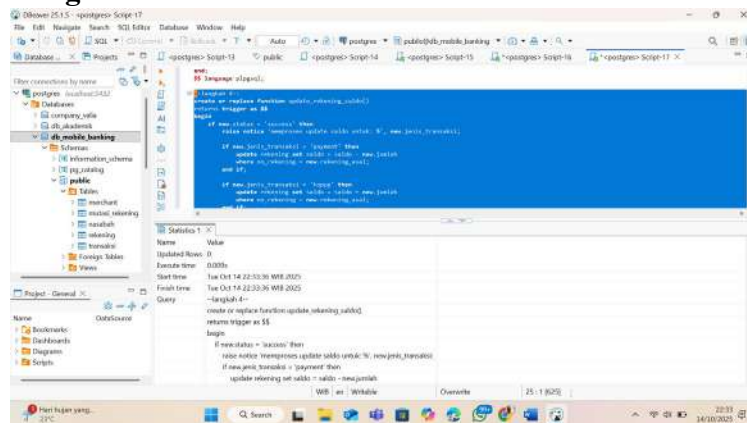
Langkah 2



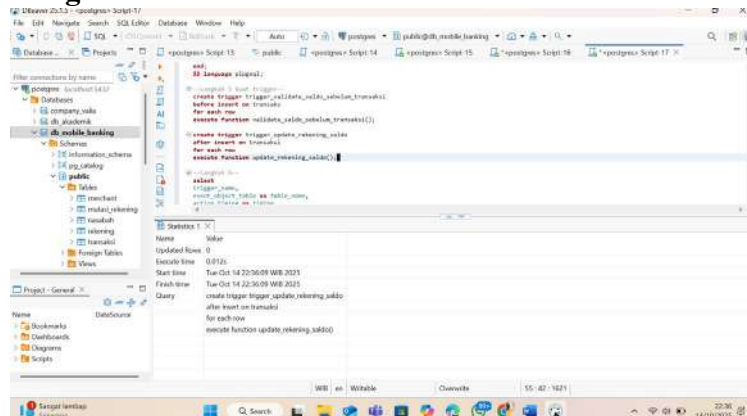
Langkah 3



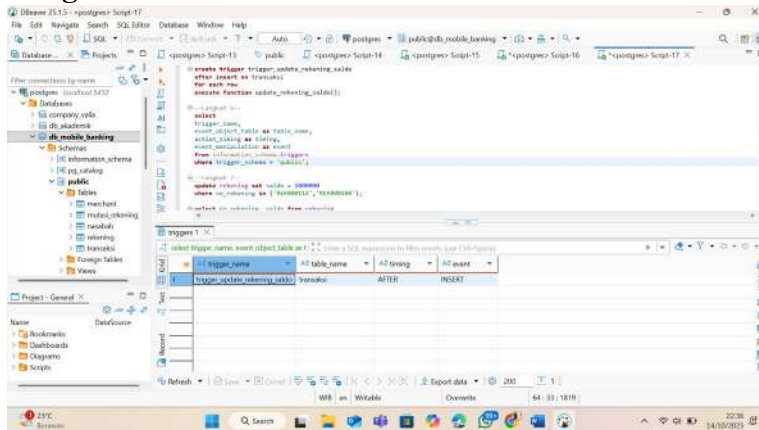
Langkah 4



Langkah 5

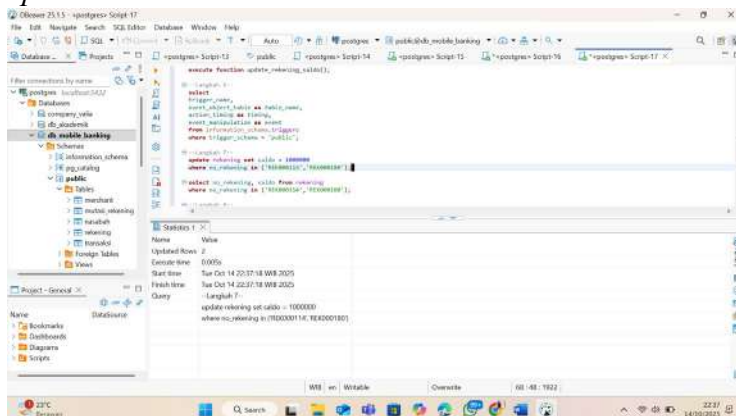


Langkah 6

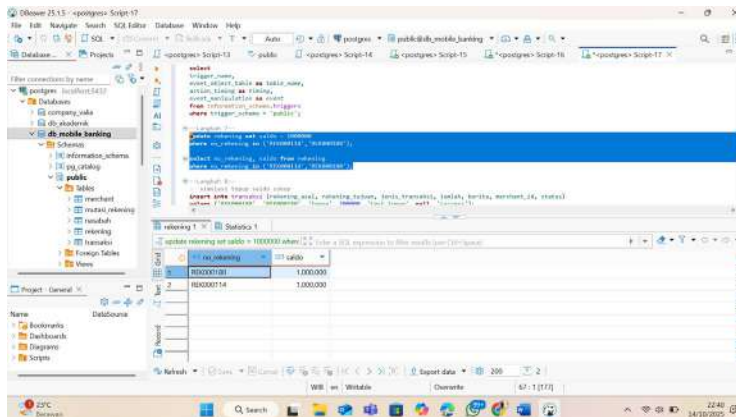


Langkah 7

update

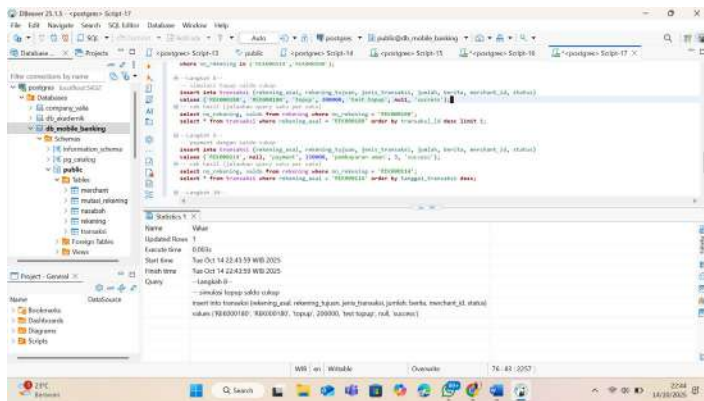


Select

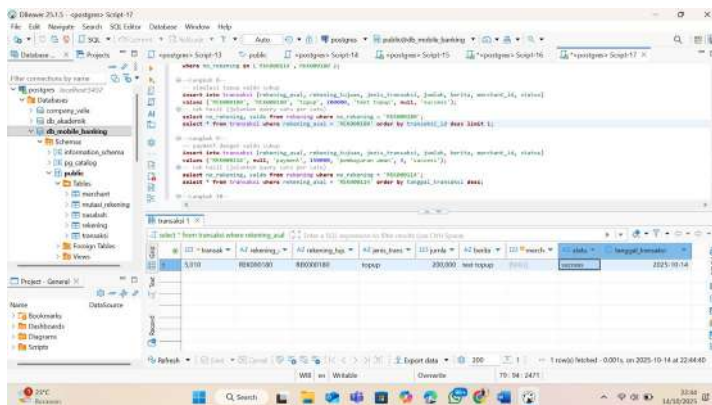


Langkah 8

Simulasi saldo cukup

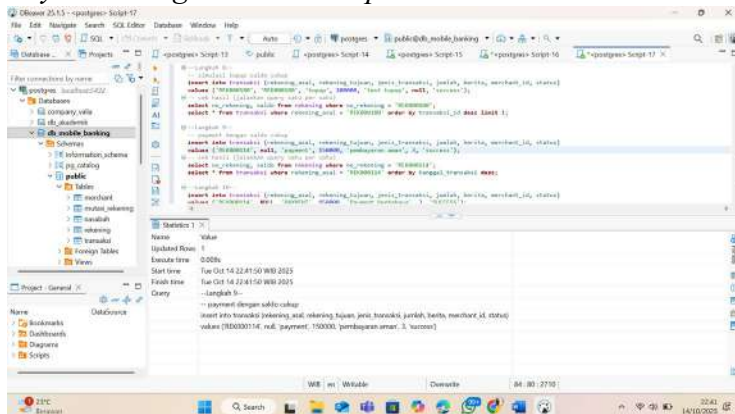


Select

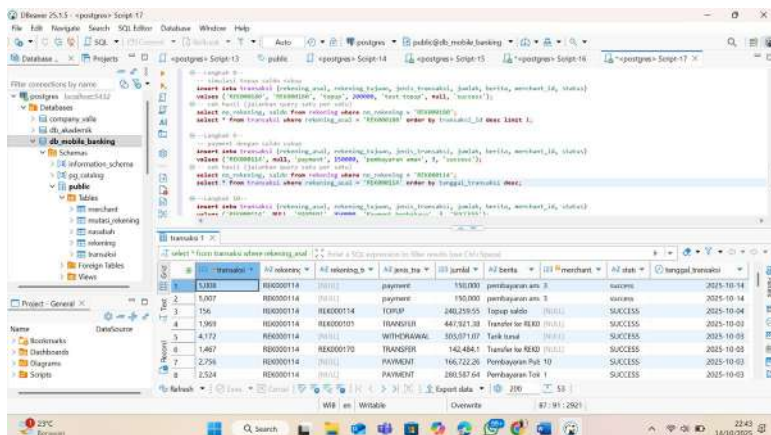


Langkah 9

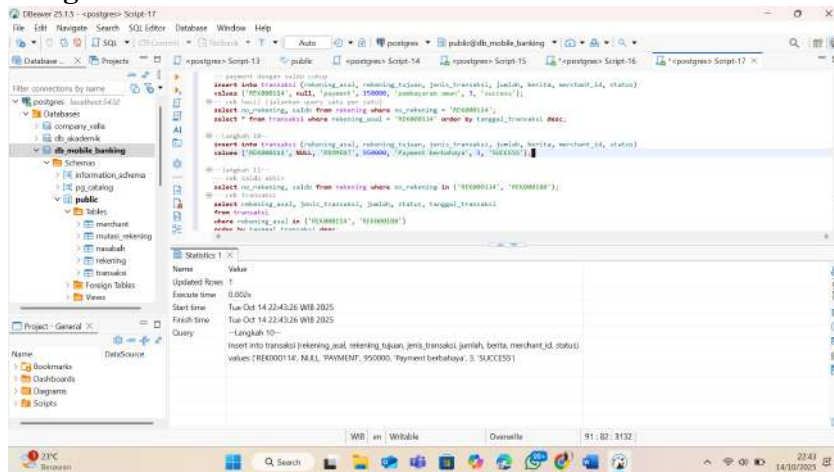
Payment dengan saldo cukup



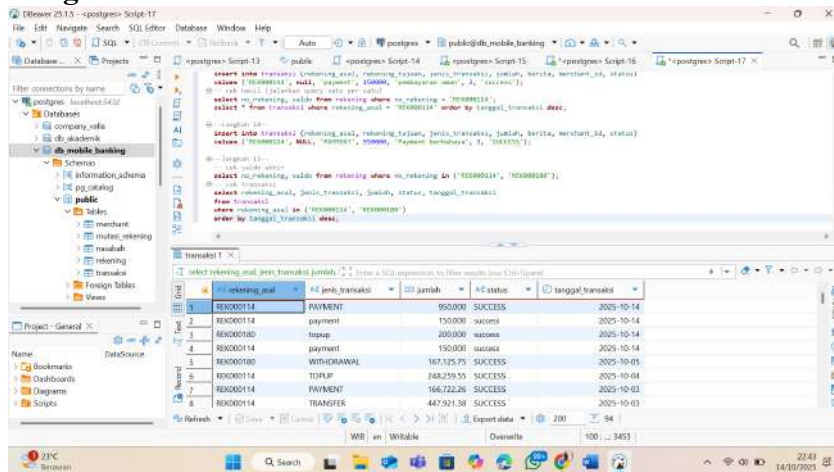
Select



Langkah 10



Langkah 11



12. Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
6	Cek trigger aktif			Trigger berhasil dibuat dan aktif di table transaksi
7	Set saldo awal	REK000114: 1,000,000 REK000180: 1,000,000	REK000114: 1,000,000 REK000180: 1,000,000	Saldo awal kedua rekening diatur menjadi 1.000.000
8	Cek saldo topup berhasil		REK000114: 1,000,000 REK000180: 1,200,000	Transaksi top up sebesar 200.000 berhasil, dan saldo bertambah menjadi 1.200.000 pada REK000180
	Cek transaksi topup berhasil			Data transaksi baru berhasil disimpan ke table

				transaksi
9	Cek saldo payment berhasil		REK000114: 850,000 REK000180: 1,200,000	Transaksi payment sebesar 150.000 berhasil, saldo REK000114 berkurang menjadi 850.000
	Cek transaksi payment berhasil			Transaksi payment tercatat di table transaksi
10	Payment gagal			Transaksi payment sebesar 950.000 gagal karena saldo tidak cukup.
11	Cek saldo akhir		REK000114: 850,000 REK000180: 1,200,000	Saldo akhir tidak berubah karena transaksi gagal tidak diproses.
	Cek transaksi akhir			Sistem hanya mencatat transaksi valid (success), transaksi gagal ditolak trigger before insert

13. Pertanyaan Analisis

a. Apa perbedaan fundamental antara trigger BEFORE dan AFTER dalam konteks praktikum ini?

Jawaban: Perbedaan utama antara trigger BEFORE dan AFTER adalah waktu eksekusinya. Jika Trigger Before dijalankan sebelum data benar benar dimasukkan ke dalam table, sedangkan AFTER dijalankan setelah data berhasil disimpan.

● Mengapa validasi saldo menggunakan BEFORE INSERT?

Jawaban: Karena sistem harus memastikan terlebih dahulu bahwa saldo rekening mencukupi sebelum data transaksi benar-benar disimpan. Jika langsung menggunakan AFTER, maka data transaksi akan terlanjur masuk ke tabel, baru dicek kemudian dan ini bisa menimbulkan inkonsistensi data (transaksi tercatat tapi saldo tidak cukup).

● Mengapa update saldo menggunakan AFTER INSERT?

Jawaban: Karena update saldo hanya boleh dilakukan jika transaksi sudah dinyatakan valid dan tersimpan. Jika trigger AFTER dijalankan, itu berarti INSERT transaksi sudah berhasil dilakukan tanpa error, artinya sistem bisa dengan aman.

b. Buat flowchart yang menunjukkan execution flow dari insert transaksi hingga update saldo, termasuk semua trigger yang terlibat!

